24 MANTRA ORGANIC × BLINKIT · REVENUE GROWTH MANAGEMENT

Operations Manual

How the discount engine runs, learns, and proves itself

Raw platform exports become a clean fact table. Two independent engines — one causal and per-cell, one portfolio-wide and cross-price — each recommend. The weekly tracker executes only what both endorse, in small guarded steps, records every prediction, backfills what actually happened, and automatically reverts anything that hurts. Every week of operation is also an experiment that grades the model.

CONTENTS

1 The system on a page
2 Architecture: five layers, one loop
3 Two engines and the quadruple lock
4 The weekly operating cadence
5 The 4-weekly refresh and its gate
6 The learning flywheel
7 The evidence ladder and geo-tests
8 Decision rules and thresholds
9 Data contracts
10 Failure modes
11 Health metrics and glossary

EXHIBIT 1

Four business questions map to four subsystems — nothing exists without a question

A cell (one SKU in one city) is the atomic decision unit. 84 SKUs × 11 cities = 585 cells today.

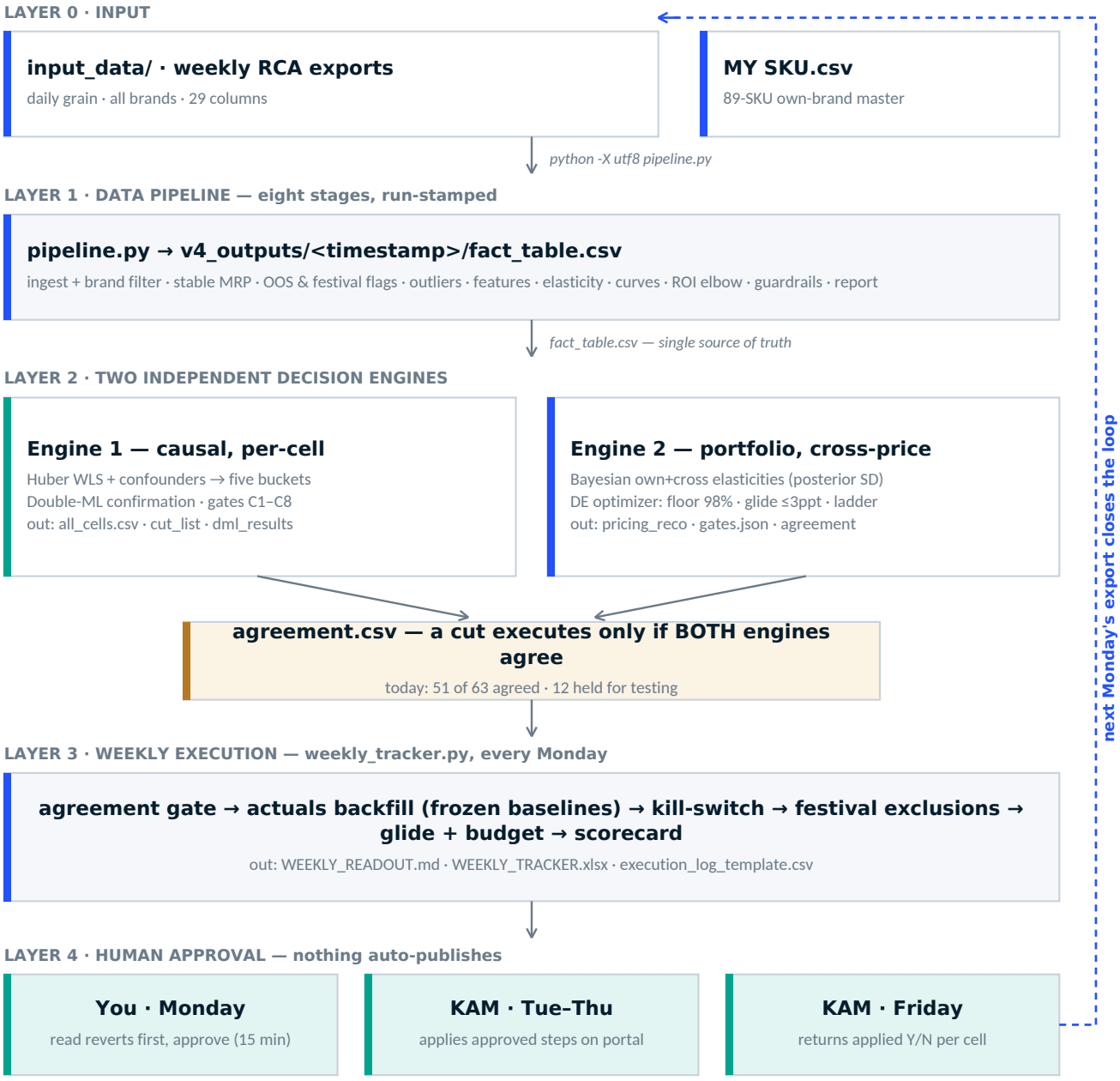
	Business question	Subsystem that answers it	Today's answer
1	Where is discount over-invested — paying for volume that would come anyway?	Causal engine: confounder-controlled regression → buckets → Double-ML confirmation → cut list	63 cells, ₹6.98L/month, 10/10 categories DML-confirmed
2	Where would more discount pay?	Reinvest detection (CI test + DML headroom) → pilot protocol	Oil & Salt headroom surfaced; pilots pending actuals
3	How should a fixed budget spread across cells?	Marginal-ROI ladder + waterline allocator under budget, glide, ladder, floor constraints	At a 10% cap the waterline sits at ROI = 1.00; profit-optimal discount is near zero
4	Is the model right week after week — and does it improve?	Weekly tracker: frozen baselines → actuals → scorecard → kill-switch → 4-weekly retrain	Loop verified end-to-end; first live actuals arrive with W2

TAKEAWAY — Prediction is an input, optimization is the product, and human approval is the gate — the same division of labor PepsiCo's PricingAI documents at enterprise scale.

EXHIBIT 2

Five layers, one closed loop: exports in on Monday, graded plans out by lunch

Data flows down; the dashed line is next Monday's export closing the loop. Every layer hands one artifact to the next.

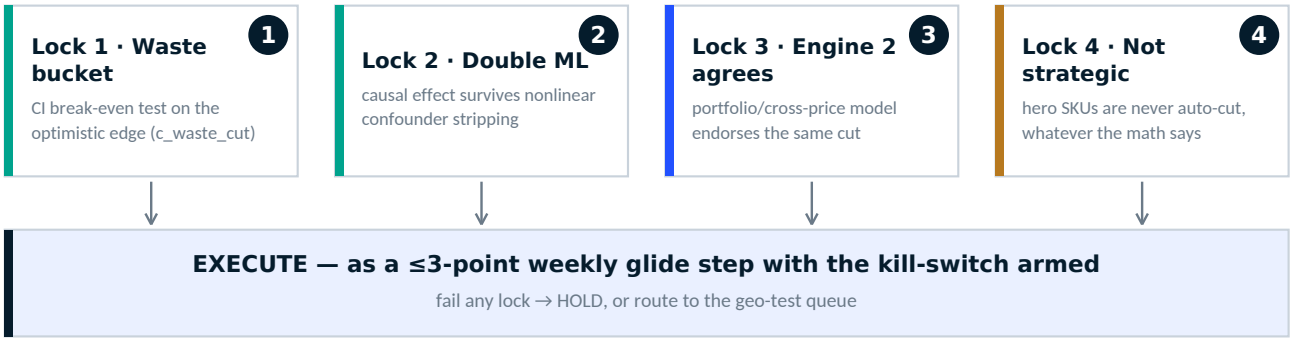


TAKEAWAY — Two engines that can each be fooled differently, plus a human gate, is deliberate redundancy — the cockpit principle: never fly on one instrument.

EXHIBIT 3

A cut needs four independent locks before a single rupee of discount moves

Each lock fails for a different reason; only cells that clear all four are executed — the rest are held or routed to tests.

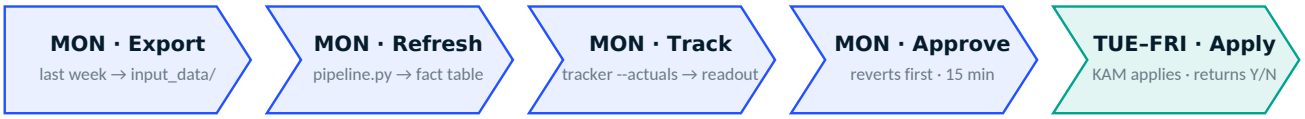


TAKEAWAY — Locks make the system slow to act and hard to fool — the trade PepsiCo's paper endorses: trust is won by what you refuse to do without evidence.

EXHIBIT 4

The week runs itself: three commands Monday, one file back Friday

Machine time ~35 minutes; your time ~15. Models are not refit weekly — execution is weekly, learning is 4-weekly.



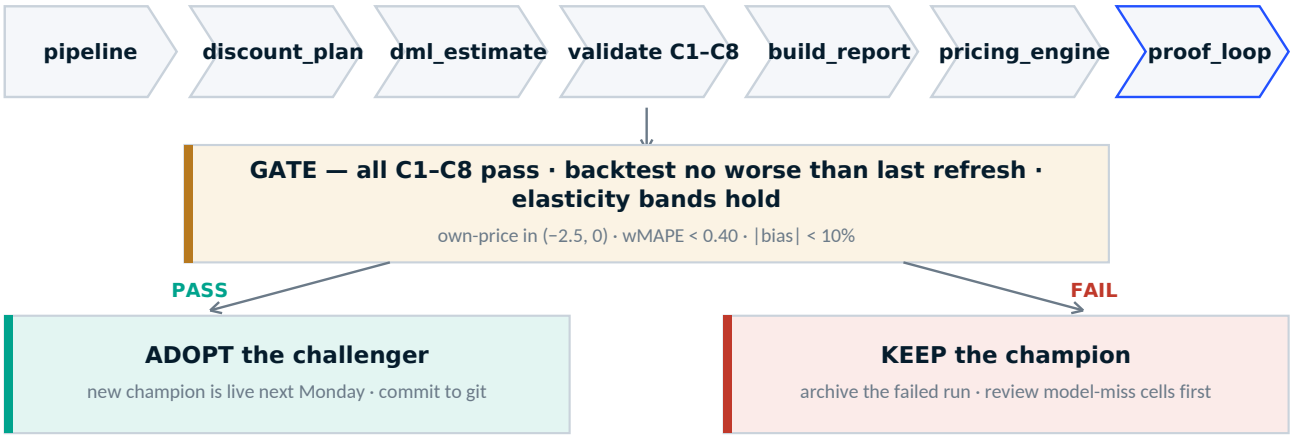
Commands: python -X utf8 pipeline.py → python -X utf8 scripts/tracker/weekly_tracker.py --actuals v4_outputs/<new>/fact_table.csv --budget_pct 0.125

Readout order: REVERT/ALERT block → budget status → this week's cuts → track record. A revert outranks every other line on the page.

EXHIBIT 5

Every four weeks the models must re-earn their job — or the champion stays

Seven steps, one gate. A challenger that cannot beat the champion out-of-sample never touches production.



TAKEAWAY — The competitor challenger already ran under this exact rule: Model B (competition-controlled) scored R^2 0.762 vs the champion's 0.781 — champion kept, and the ₹6.98L survived at −5%.

EXHIBIT 6

Six moves, one flywheel: every week of operation grades the model that planned it

Frozen baselines make the grading honest; the kill-switch makes mistakes cheap; the retrain makes lessons permanent.

EXHIBIT 7

Evidence comes in rungs; climb only as high as the money at stake demands

Regression screens everything weekly for free. Controlled tests settle the few expensive questions. Constraints keep every answer safe regardless.

EXHIBIT 8

The geo-split test in one picture: the control cities are the world where you did nothing

One category, one change, matched city pairs split by coin flip, three weeks, decided by a rule written before the test starts.

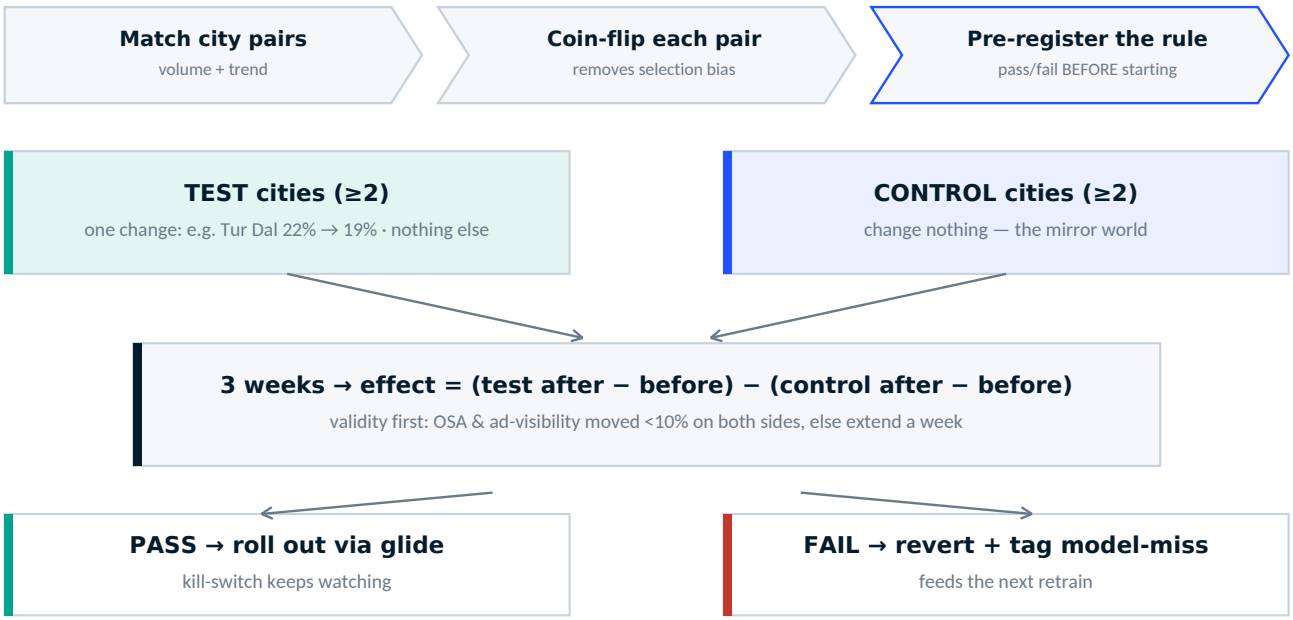


EXHIBIT 9

Every threshold is one sentence with a number — and every number has a reason

These live in config and rule files, not buried in code. Change them there, never inline.

Rule	Value	Why this value
Weekly glide step	≤ 3 ppt per cell	small enough to revert cheaply; reaches any target in ~12 weeks
Revenue-protective revert	predicted Δ < 0 → hold	never take a step the model itself predicts loses money
Budget cap	--budget_pct (set the real number)	blocks reinvest increases above cap; bands GREEN ≤95 · AMBER 95–105 · RED >105
Kill-switch strike	units < pred × 0.95 AND net-rev Δ < 0	both legs required — a cheap miss that still makes money is not a failure
Confounder excuse	OSA or ad-visibility < 90% of baseline	an out-of-stock week says nothing about the discount
Revert + freeze	2 consecutive strikes → restore, freeze 4 wks	one bad week is noise; two is a pattern; freeze stops thrashing
Drift brake	hit-rate < 60% over ≥ 30 scored cells	if aim is off portfolio-wide, stop new cuts and force a retrain
Waste test	optimistic CI edge below break-even 1/(100−d)	only bank what survives the favourable reading
Reinvest test	pessimistic CI edge above break-even + headroom	symmetric honesty before spending more
Elasticity gates	own (−2.5, 0) · wMAPE < 0.40 · |bias| < 10%	PepsiCo's published acceptance bands
DML confirmation	θ + 1.96·SE below break-even, per category	cuts must survive nonlinear-confounder stripping
DE constraints	revenue ≥ 98% · ₹/kg ladder · PPP · box 0–45%	portfolio safety fences
Festival weeks	excluded from waste scoring; +50% budget allowance	planned festive discounts are strategy, not waste
Geo-test decision	pre-registered DiD rule · 3 weeks · ≥2+2 cities	evidence, not vibes

EXHIBIT 10

Twelve files carry the whole system — each one a handoff with an owner

If a file is missing or malformed, the consumer column tells you exactly what breaks.

File	Content / grain	Producer → Consumer
input_data/*_RCA.csv	product × city × day · all brands · 29 cols	you → pipeline stage 1
MY SKU.csv	89-row own-brand allowlist + pack sizes	you → stage 1 brand filter
fact_table.csv	cell × day, cleaned: stable MRP, real discount, OOS/festival/outlier flags	stage 2 → every model
plan/all_cells.csv	one row per cell: bucket, confidence, current vs target discount, reason	Engine 1 → tracker + Engine 2
plan/dml_results.json	per-category causal θ, SE, verdict	Engine 1 → gate C8 + reinvest
pricing/agreement.csv	per cell: does Engine 2 endorse the cut?	Engine 2 → tracker gate
pricing/gates.json	model quality certificate (R^2, wMAPE, bias, bands)	Engine 2 → you
pricing/roi_ladder.csv	every cell × discount step → units, net-rev, spend, marginal ROI, elbow	allocator → brand-facing proof
tracker_history.csv	cell × week: predictions, actuals, applied, strikes, status	tracker → scorecard + kill-switch
baselines.json	frozen pre-action baseline per cell (never recomputed)	actuals.py → all grading
execution_log.csv	week, cell, applied Y/N — KAM's Friday truth	KAM → tracker scoring
WEEKLY_READOUT.md / .xlsx	the weekly decision document	tracker → you + KAM

TAKEAWAY — Baselines are frozen on purpose: if the “before” number moved every week, natural drift would fake wins and losses. One yardstick per cell, captured at action start, never overwritten.

EXHIBIT 11

Ten known failure modes — each with its cause and its one-line fix

Everything below has actually been seen (or deliberately designed for). Pin this page.

Symptom	Cause	Fix
SyntaxError importing tracker modules	cloud-sync truncated files mid-write	git status → git restore → re-run smoke tests
“No all_cells.csv” from tracker	Engine 1 chain not run	run discount_plan → dml → validate first
“scored cells: 0” after week 2	execution_log.csv missing or still named _template	fix the KAM file; values must be Y/N
A week seems skipped	same label already logged (idempotent append)	by design — delete that week's rows to redo
Everything flagged confounded	platform-wide availability collapse	correct behaviour — fix OSA before judging discount
Drift brake fires immediately	scoring unacted cells (pre-fix)	fixed at 82ea05a; verify with killswitch smoke test
Gates fail at a refresh	thin or odd new data	champion stays; inspect per-category coverage
Cells collapse to a handful	over-broad brand pattern	fix OWN_BRAND_PATTERNS; re-run pipeline
Engines disagree on many cells	confounded categories	designed: those route to tests, not cuts
₹ / → print as boxes	terminal encoding	always run with python -X utf8

EXHIBIT 12

Six numbers tell you if the system is healthy — review monthly, show the brand quarterly

Hit rate
predicted direction correct
trend toward ~85%

Decision MAPE
error at 3-ppt bins
stable or falling

Revenue bias
systematic over-promise?
within ±10%

Realized ₹ vs claim
the register vs ₹6.98L/mo
the only number brands feel

% applied
ops health, not model health
below 70% = broken KAM loop

Reverts / month
model health
a few = learning; a spike = retrain

THE OPERATION IN ONE LINE — Monday: data in, plan out. Friday: KAM's Y/N in. Every four weeks: models re-earn their job. Every claim: gated, agreed by two engines, glided in 3-point steps, watched by a kill-switch, graded by the register.